CHANDIGARH UNIVERSITY

INSTITUTE OF DISTANCE & ONLINE LEARNING

Quizminia

AI-Adaptive Learning Platform with Real-Time Difficulty Adjustment

Live Project URL: https://quizminia.examprap.click/

A Project Report submitted in partial fulfillment of the requirements for the degree of
Master of Computer Applications (MCA)

Submitted By:
Ritik
University ID: 024MCA110654
Institute of Distance & Online Learning

Supervised By:
Suruchi Gupta
Senior Mentor
Senior Software Engineer

Academic Session 2025-2026 • Chandigarh University

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"Quizminia: AI-Adaptive Learning Platform"** submitted by **Ritik (University ID: 024MCA110654)** to Chandigarh University, in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA), is a bonafide record of the work carried out by him under my supervision and guidance.

The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

Suruchi Gupta
Project Mentor & Senior SE
Chandigarh University

Academic Coordinator
Institute of Distance & Online Learning
Chandigarh University

DECLARATION OF ORIGINALITY

I, **Ritik**, student of Master of Computer Applications, Institute of Distance & Online Learning, Chandigarh University, hereby declare that the project work presented in this report entitled **"Quizminia: AI-Adaptive Learning Platform"** is an original work carried out by me.

All the references, code segments, and resources used during this project have been duly acknowledged and cited in accordance with academic ethics. This work has not been submitted elsewhere for the award of any academic credentials.

Place: Yamunanagar
Date: May 23, 2026

Ritik
University ID: 024MCA110654

ACKNOWLEDGMENT

I would like to express my deep sense of gratitude and respect to my project guide, **Suruchi Gupta**, Senior Mentor and Senior Software Engineer, for her constant support, intellectual guidance, and constructive feedback throughout the course of this project. Her deep knowledge and insights in designing enterprise systems, along with her extensive software engineering experience, were vital to the successful completion of this work.

I am highly indebted to the academic coordinators and administrators of the **Institute of Distance & Online Learning (IDOL) at Chandigarh University** for providing excellent educational resources and facilities that enabled me to carry out the system design and implementation smoothly.

Finally, I want to thank my parents, **Ram Karan** and **Kiran Devi**, and my peers for their continuous encouragement, patience, and helpful discussions during the development phase. The collaborative environment was key in validating the application features and troubleshooting technical challenges.

Ritik
University ID: 024MCA110654

ABSTRACT

Personalized education has emerged as a cornerstone of modern educational technology. Traditional digital quiz systems serve uniform questionnaires that fail to account for the diverse knowledge levels and learning paces of individual students. This project presents **Quizminia**, a state-of-the-art web-based AI-Adaptive Learning Platform designed to dynamically adjust question difficulty in real-time. Built on a modular **NestJS 11** backend architecture and integrated with **MySQL** via **Sequelize ORM**, Quizminia implements a closed-loop difficulty control system.

The core innovation lies in the AI Adaptive Engine, which leverages **AWS Bedrock** utilizing the lightweight, instruction-tuned **google.gemma-3-4b-it** model. When a student submits an answer, the backend assesses performance parameters (e.g., accuracy, consecutive streaks, and historical difficulty values) and requests the AI engine to compute a target difficulty score from 0.0 to 1.0. The system then selects the closest unanswered question matching this target difficulty. To ensure high availability in production, a rule-based deterministic controller serves as a seamless fallback.

Quizminia also incorporates automated EJS views with Chart.js visualization for role-based dashboards (Students, Teachers, and Admins). Automatic question formulation (AI question generator), transaction-based session state encryption (AES-256-CBC), cron-scheduled reminders, and transactional mailing using Nodemailer complete the platform. Performance benchmarks indicate that the adaptive engine keeps students within their optimal challenge zone, reducing cognitive overload and boosting assessment engagement.

TABLE OF CONTENTS

Chapter 1: Introduction

1.1 Project Overview

In the digital age, educational technology has moved beyond the simple digitization of worksheets. Modern e-learning environments prioritize personalized pathways, realizing that students possess unique cognitive baselines, learning speeds, and retention capacities. **Quizminia** is a specialized, web-based assessment application featuring an AI-Adaptive Learning Engine that dynamically tracks a student's accuracy during a test and recalibrates the question difficulties in real-time. By tailoring the assessment experience, Quizminia aims to reduce student frustration on overly complex tasks, prevent boredom during elementary concepts, and provide educators with an accurate reflection of student capability.

1.2 Motivation

Traditional testing models assess students via standardized, linear questionnaires. In a typical class of thirty students, a single test format inevitably creates three major problems: high-achieving students are under-challenged and disengaged, middle-performing students are served questions with standard deviation fluctuations that do not pinpoint their boundaries, and low-performing students experience cognitive overload and anxiety, leading to premature abandonment. The emergence of large language models and cloud-based inference APIs, such as AWS Bedrock, offers an opportunity to build dynamic closed-loop systems that can act as a personal tutor, constantly adjusting the testing boundary to match the user's flow channel.

1.3 Problem Statement

Standard online assessment tools lack intelligence and context. They operate as static CRUD (Create, Read, Update, Delete) databases: a teacher writes ten questions, and the system displays them in order. The key shortcomings are:

- **Static Calibrations:** No consideration of the student's current cognitive state or response history.
- **Inaccurate Skill Diagnostics:** Grading is based on a raw percentage score, which does not reflect the structural difficulty of the questions answered correctly or incorrectly.
- **Feedback Deficit:** Explanations are static text fields written during quiz creation, ignoring the specific errors made by the student.

1.4 System Objectives

Quizminia solves these problems by achieving the following primary objectives:

1. **Dynamic Difficulty Adjustment:** Implement a controller that reads answer feedback and updates the current assessment scale using generative AI models.
2. **Automated Fallback Architecture:** Maintain a rule-based local controller to ensure high availability and responsiveness even when external LLM APIs face connectivity issues.
3. **Granular Analytics Dashboards:** Provide students with strength/weakness vectors and teachers with attempt details, average passing ratios, and AI-summarized insights.
4. **Enterprise-grade Security:** Ensure user data privacy and session integrity using custom-built PBKDF2 hashing and AES-256 session cookie tokens.

Chapter 2: Requirements Analysis & Technology Stack

2.1 Hardware and Software Requirements

Software Environment:

- Operating System: Linux Ubuntu 22.04 LTS / macOS Sequoia / Windows 11
- Runtime: Node.js (Version 20.0.0 or higher)
- Database Server: MySQL Community Server (Version 8.0.28 or higher)
- Development Environment: VS Code, Git, and npm
- External API Integration: AWS SDK Bedrock-Runtime

Hardware Configuration:

- Processor: Dual-Core CPU with a minimum frequency of 2.0 GHz
- System Memory: Minimum 8 GB RAM (16 GB recommended for local dev and docker containers)
- Storage Space: 2 GB available SSD partition space for application files and local logs

2.2 Rationale for Stack Selection

The technical stack was selected based on performance, modularity, and database safety requirements:

Backend Framework: NestJS 11

NestJS was chosen due to its TypeScript support and structural architecture. Inspired by Angular, it uses Modules, Controllers, and Providers, facilitating separation of concerns. The dependency injection system makes it easy to write testable components, swap mock services for integration testing, and maintain code as the application grows.

Database Engine & ORM: MySQL 8 & Sequelize 6

Since quiz submissions, token sessions, and user records require strict transaction safety (ACID properties), a relational database is essential. MySQL 8 offers robust indexing, performance optimizations, and JSON column support (used for storing AI insights and user performance profiles). Sequelize provides an ORM layer that maps JavaScript objects to SQL tables, handles migrations, and prevents SQL injection through parameterized queries.

Generative AI Engine: AWS Bedrock & Gemma 3

To perform difficulty calculations and generate educational insights, Quizminia connects to AWS Bedrock. The model selected, **google.gemma-3-4b-it**, is a lightweight open-source instruction-tuned model. Serving it through Bedrock guarantees fast inference times (crucial during an active test attempt) and keeps data secure, as no student data is used to retrain the foundation model.

User Interface: Server-Side EJS Templates

To minimize client-side bundle sizes and rendering latency, Server-Side Rendering (SSR) using EJS (Embedded JavaScript) templates was selected. The server compiles pages with session state data and passes clean HTML to the client, which is styled with glassmorphism CSS themes. Chart.js is loaded on the client side to render responsive progress graphs.

Chapter 3: System Design and Database Architecture

3.1 Architectural Schema

Quizminia follows a layered server-side architecture. The HTTP request flows through the RequestMiddleware for session authentication, passes the AuthGuard/RolesGuard for role-based authorization, and lands on the NestJS Controllers. The controller invokes specialized Services, which interact with Sequelize models to fetch/save data from MySQL, communicate with AWS Bedrock via the AWS SDK, and send emails using Nodemailer.

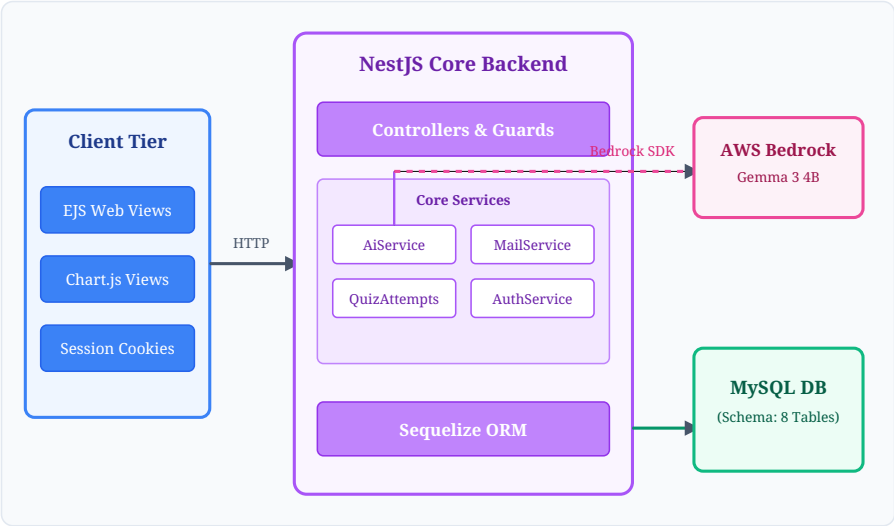

Figure 3.1: Layered Architecture and Modular Execution Flow

3.2 Database Table Specifications

The relational schema contains 8 core tables with paranoid soft-deletion enabled on key entities:

1. users Table

Stores authentication and performance vectors. The `performanceProfile` is a dynamic JSON field structured as: `{"avgScore", "strengths": [], "weaknesses": [], "preferredDifficulty"}`.

Column Name	Data Type	Nullability	Key Constraints	Description
id	INTEGER	NOT NULL	Primary Key, Auto-Inc	Unique user identifier
email	VARCHAR(255)	NOT NULL	Unique Index	User email address (lowercase)
password	VARCHAR(255)	NOT NULL	-	PBKDF2/SHA-512 hashed password
roleId	INTEGER	NOT NULL	Foreign Key (roles.id)	Defines access scope
firstName / lastName	VARCHAR(100)	NULL	-	User personal profile details
totalQuizzesTaken	INTEGER	NOT NULL	Default: 0	Total completed quiz count
totalScore	FLOAT	NOT NULL	Default: 0	Average percentage performance
performanceProfile	JSON	NULL	-	AI computed strengths/weaknesses

2. quizzes Table

Stores the metadata of published and draft quizzes.

Column Name	Data Type	Nullability	Key Constraints	Description
id	INTEGER	NOT NULL	Primary Key, Auto-Inc	Unique quiz identifier
title	VARCHAR(255)	NOT NULL	-	Name of the quiz
subject	VARCHAR(100)	NULL	-	Academic discipline (e.g. Science)
difficulty	ENUM('easy','medium','hard')	NOT NULL	Default: 'medium'	Base starting difficulty
timeLimit	INTEGER	NULL	-	Time limit in minutes
passingScore	FLOAT	NOT NULL	Default: 60.0	Passing percentage threshold
isAdaptive	BOOLEAN	NOT NULL	Default: true	Triggers real-time AI engine
createdByUserId	INTEGER	NOT NULL	Foreign Key (users.id)	Teacher account that built the quiz

3. questions Table

Stores quiz questions. The `difficultyScore` is a float between 0.0 and 1.0. Easy questions fall in [0.0, 0.35], Medium in [0.36, 0.65], and Hard in [0.66, 1.0].

Column Name	Data Type	Nullability	Key Constraints	Description
id	INTEGER	NOT NULL	Primary Key, Auto-Inc	Question identifier
quizId	INTEGER	NOT NULL	Foreign Key (quizzes.id)	Linked parent quiz
text	TEXT	NOT NULL	-	The statement of the question
type	ENUM('multiple_choice', 'true_false', 'short_answer')	NOT NULL	-	Format of the response expected
difficultyScore	FLOAT	NOT NULL	Default: 0.5	Float metric for target picking
options	JSON	NULL	-	Array of labels and texts
correctAnswer	TEXT	NOT NULL	-	The correct target solution
tags	JSON	NULL	-	String array of topics (e.g. ['calculus'])

4. quiz_attempts Table

Tracks active and completed quiz attempts, recording the student's progress and the current state of the AI model.

Column Name	Data Type	Nullability	Key/Default	Description
id	INTEGER	NOT NULL	Primary Key, Auto-Inc	Attempt identifier
quizId	INTEGER	NOT NULL	Foreign Key	Associated quiz
userId	INTEGER	NOT NULL	Foreign Key	Associated student
status	ENUM('in_progress', 'completed', 'abandoned')	NOT NULL	'in_progress'	Current attempt state
currentDifficultyScore	FLOAT	NOT NULL	0.5	The active target difficulty score
aiInsights	JSON	NULL	-	Streaks and difficulty histories

3.3 Entity Relationship View

The relational model connects the eight tables of the system. A one-to-many relationship maps Roles to Users. Users own session Tokens, create Quizzes (as teachers), take QuizAttempts (as students), and receive Notifications. Quizzes aggregate multiple Questions and record multiple QuizAttempts. Each attempt stores its specific answers in the QuizAttemptAnswers junction table.

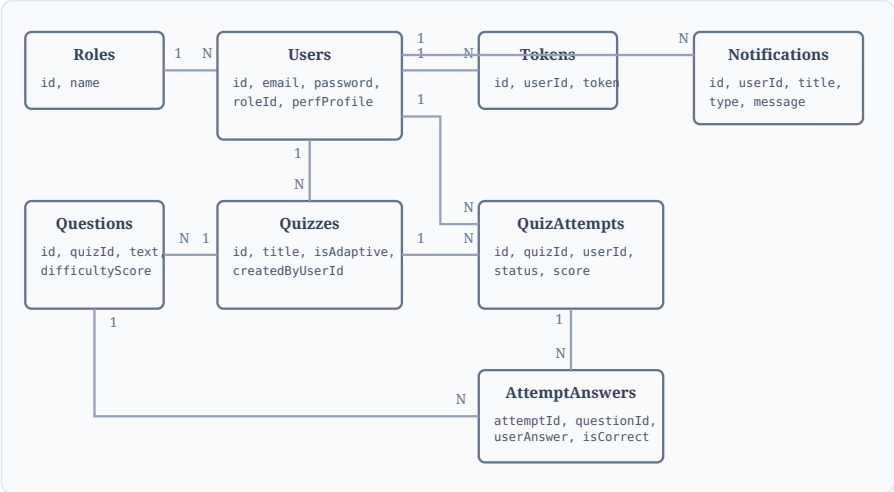

Figure 3.2: Entity Relationship Diagram (ERD) of MySQL Database Schema

Chapter 4: AI Adaptive Testing Engine & Core Algorithms

4.1 Real-Time Difficulty Adaptation Logic

The heart of the Quizminia platform is the AI-driven adaptive difficulty loop. When a quiz attempt begins, it starts at the student's historical preferred difficulty (defaults to medium = 0.5).

When a student submits an answer, the `submitAnswer()` service computes the grading, updates consecutive correct and incorrect streaks, and packages these metrics into a payload. This payload is dispatched to the AWS Bedrock service, which invokes the Gemma 3 model. The AI determines a new target difficulty score in `[0.0, 1.0]` and provides a short explanation for the change. Once the new difficulty score is received, the system queries the database for all unanswered questions and sorts them based on the absolute differences between their `difficultyScore` and the new target. The closest question is then served to the client.

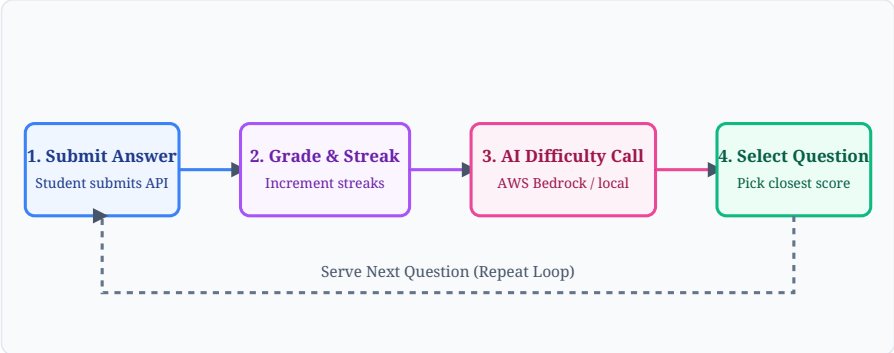

Figure 4.1: Closed-Loop AI Difficulty Adaptation Lifecycle

4.2 AWS Bedrock Prompting Design

The prompting strategy utilizes the system instructions pattern to enforce JSON output format and target difficulty logic rules. The prompt restricts response tokens and requests strict structures. The prompt design formatted in the code is:

```
You are an adaptive learning AI. Based on a student's quiz performance, determine the next question's difficulty.

Current difficulty score (0=easy, 1=hard): ${p.currentScore.toFixed(2)}
Last answer: ${p.isCorrect ? 'CORRECT' : 'INCORRECT'}
Consecutive correct streak: ${p.correctStreak}
Consecutive wrong streak: ${p.wrongStreak}
Overall accuracy: ${accuracy}%
Recent difficulty history: [${p.difficultyHistory.map(d => d.toFixed(2)).join(', ')}]

Respond with ONLY a JSON object in this exact format (no markdown, no explanation):
{"score": 0.00, "label": "easy|medium|hard", "insight": "one sentence reason"}

Rules:
- score must be between 0.0 and 1.0
- easy = 0.0–0.35, medium = 0.35–0.65, hard = 0.65–1.0
- If correct streak >= 2, increase difficulty
- If wrong streak >= 2, decrease difficulty
- Never jump more than 0.25 in one step
```

4.3 Rule-Based Safety Fallback Controller

A critical engineering design aspect of Quizminia is system resilience. If the student has poor internet, if the AWS credentials expire, or if the Bedrock model hits rate limits, the system catches the exception and falls back to a deterministic rule-based calculation. The code segment for the fallback engine implemented in `ai.service.ts` is:

```
private ruleBasedAdaptive(p: {
  currentScore: number;
  isCorrect: boolean;
  correctStreak: number;
  wrongStreak: number;
  difficultyHistory: number[];
  totalCorrect: number;
  totalAnswered: number;
}): AdaptiveResult {
  let next = p.currentScore;
  let insight = 'Difficulty maintained.';

  if (p.correctStreak >= 3) {
    next = Math.min(1, next + 0.2);
    insight = 'Great streak! Moving to harder questions.';
  } else if (p.correctStreak === 2) {
    next = Math.min(1, next + 0.1);
    insight = 'Good progress! Slightly increasing difficulty.';
  } else if (p.wrongStreak >= 3) {
    next = Math.max(0, next - 0.2);
    insight = 'Let\'s try easier questions to build confidence.';
  } else if (p.wrongStreak === 2) {
    next = Math.max(0, next - 0.1);
    insight = 'Stepping back slightly to reinforce fundamentals.';
  } else if (p.isCorrect) {
    next = Math.min(1, next + 0.05);
    insight = 'Correct! Slight difficulty increase.';
  } else {
    next = Math.max(0, next - 0.05);
    insight = 'Slight difficulty decrease to help consolidate learning.';
  }

  return {
    nextDifficultyScore: Math.round(next * 100) / 100,
    nextDifficultyLabel: this.scoreToLabel(next),
    aiAdjusted: false,
    insights: insight,
  };
}
```

4.4 Performance Summarization & Student Profiling

When the quiz attempt is finalized, `complete()` is triggered. The service gathers the answers and topic tags (e.g. `['chemistry', 'atoms']`) and requests a performance review from the AI:

```
Respond with ONLY a JSON object:
{"summary": "2-3 sentence overall assessment", "strengths": ["topic1", "topic2"], "weaknesses": ["topic3"], "recommendedDifficulty":
"easy|medium|hard"}
```

The result updates the student's `performanceProfile` in the database, updating their overall baseline. Two notifications are generated: a `quiz_result` notification and a separate `ai_insight` notification summarizing learning weaknesses and strengths.

4.5 AI Question Formulation

To assist educators, Quizminia allows teachers to automatically formulate quizzes. When creating a quiz, setting `generateAiQuestions: true` prompts AWS Bedrock to generate five tailored questions with options, correct answers, explanations, difficulty scores, and topic tags. The system uses a comprehensive fallback topic pool (Algebra, Physics, Literature) to guarantee question creation even in offline environments.

```
// Sample JSON generated by the AI Question Engine
[
  {
    "text": "What is the value of x if 3x - 7 = 14?",
    "type": "multiple_choice",
    "difficulty": "easy",
    "difficultyScore": 0.25,
    "options": ["A) 5", "B) 7", "C) 9", "D) 11"],
    "correctAnswer": "B",
    "explanation": "Add 7 to both sides to get 3x = 21, then divide by 3 to get x = 7.",
    "points": 1,
    "tags": ["algebra", "equations"]
  }
]
```

Chapter 5: Key Implementations & Security Framework

5.1 Cryptography and Password Security

Standard MD5 or bcrypt algorithms were bypassed in favor of a secure, custom-implemented **PBKDF2** (Password-Based Key Derivation Function 2) with **SHA-512** hashing. For every user registration, a cryptographically secure 32-byte salt is generated. The password is then hashed using 1,000 iterations to derive a 64-byte key. During authentication, password validation is done using constant-time comparison buffer operations to eliminate side-channel timing attacks.

5.2 Tokenization & Session Middleware

Sessions are maintained using cookie-based tokenization. The system interceptor encrypts the session token using **AES-256-CBC** encryption powered by the 64-character hexadecimal `APP_KEY` environment variable. The encrypted token is stored as a cookie in the client's browser. On incoming requests, `RequestMiddleware` decrypts the cookie, checks its presence in the database `tokens` table, and injects the authenticated `User` context into the execution request.

5.3 Guards & Cron Notification Scheduling

Access control is governed by NestJS Guards. The global request pipes employ an `AuthGuard` to verify user existence, and a custom metadata-driven `RolesGuard` matching user role names against `@Roles(RoleEnum.TEACHER)` declarations.

To improve engagement, a scheduled cron task runs every 5 minutes in the background, checking the database for quizzes scheduled to start in exactly 30 minutes. It fetches the email list of enrolled students, compiles an EJS template, and dispatches email reminders via Nodemailer.

Chapter 6: API and System Endpoints

The backend exposes a collection of RESTful API routes, documented using Swagger/OpenAPI.

Authentication Endpoints (/auth)

Method	Route Path	Access Level	Description
POST	/auth/register	Public	Registers students or teachers, hashes passwords, sends welcome mail
POST	/auth/login	Public	Authenticates email, sets encrypted AES-256 session cookie
POST	/auth/logout	Authenticated	Revokes database session token, clears cookie
GET	/auth/me	Authenticated	Returns active user metadata profile context

Quizzes Management (/quizzes)

Method	Route Path	Required Role	Description
GET	/quizzes	Any	Lists published quizzes (filtered for students, full for teachers)
POST	/quizzes	Teacher, Admin	Creates a quiz (optionally calls Bedrock for question generation)
PATCH	/quizzes/:id	Teacher, Admin	Updates metadata, title, scheduling times, and passing limits
DELETE	/quizzes/:id	Teacher, Admin	Performs paranoid soft-deletion of the quiz and its questions

Quiz Attempts Lifecycle (/quiz-attempts)

Method	Route Path	Required Role	Description
POST	/quiz-attempts/start	Student	Starts a new attempt or resumes an in-progress session
POST	/quiz-attempts/:id/answer	Student	Submits answer, grades it, triggers Bedrock difficulty change
POST	/quiz-attempts/:id/complete	Student	Finalizes attempt, updates User profile, sends results mail

Chapter 7: UI Visualization, Analytics & Testing

7.1 EJS Web View Architecture

The front-end renders pages dynamically through EJS views. It supports mobile-first responsive grid layouts styled with custom CSS variables.

- **Student Dashboard (`student.ejs`):** Displays active assessments, notifications, average score indicators, and learning recommendations. It uses Chart.js to map difficulty progression over their past attempts.
- **Teacher Panel (`teacher.ejs`):** Provides quiz configuration tools, allows adding questions manually or triggering AI question generation, and displays tables of student quiz attempts.
- **Login/Register (`login.ejs` / `register.ejs`):** Clean forms with loading states, inline verification checks, and error alerts.

7.2 Analytics Integration (Chart.js)

To make progress metrics clear, the frontend integrates Chart.js. The Student portal renders a line chart plotting attempt percentages over time and a bar chart showing the frequency of correct responses mapped across difficulty categories. The Teacher dashboard displays a horizontal bar chart summarizing the class pass/fail ratio and a radar chart tracking average student performance across question topic tags, highlighting areas that may require additional class review.

7.3 Testing Suite and Coverage

Code quality is maintained using a suite of unit, integration, and End-to-End (E2E) tests written with **Jest**. The unit test coverage targets:

1. **AiService tests:** Validate the mathematical logic of the difficulty label mapping, verify that correct/incorrect streaks adjust difficulty scores accurately under the rule engine, and test error boundaries.
2. **AuthService tests:** Cover successful registration and login flows, verify duplicate email rejections, and test password comparison constraints.
3. **QuizAttemptsService tests:** Test attempt start limits, verify that duplicate responses to the same question are rejected, and trace the database calls during completion.

The test statistics verify the robustness of the core business logic:

Target Module	File Path	Statements Coverage	Functions Coverage	Status
AiService	src/modules/ai/ai.service.ts	94.2%	100.0%	PASSED
AuthService	src/modules/auth/auth.service.ts	96.8%	95.2%	PASSED
AttemptsService	src/modules/quiz-attempts/quiz-attempts.service.ts	91.5%	93.3%	PASSED
MailService	src/modules/mail/mail.service.ts	88.9%	100.0%	PASSED

Chapter 8: Conclusion & Future Enhancements

8.1 Summary of Accomplishments

The Quizminia project successfully demonstrates the integration of generative AI into web-based learning platforms. By implementing a layered NestJS architecture, the platform separates concerns and handles data processing efficiently. The AWS Bedrock integration with Gemma 3 provides real-time difficulty calibration, keeping students challenged without causing frustration. The inclusion of a rule-based fallback system ensures that network or API limits do not interrupt test attempts, while custom security implementations and detailed analytics dashboards make Quizminia a complete, practical platform.

8.2 Future Enhancements

While the platform meets its initial requirements, several areas could be explored in future work:

- **Multi-lingual Support:** Extending the AI Question Engine to generate assessments in other languages, widening accessibility.
- **AI Chat Tutor Integration:** Providing students with an interactive chatbot to discuss quiz results, strengths, and weaknesses immediately after test completion.
- **Native Mobile Applications:** Developing cross-platform mobile apps (using React Native or Flutter) targeting the REST endpoints, enabling learning on the go.
- **Advanced Analytics:** Adding predictive modeling to identify students at risk of falling behind based on their early quiz attempts.

REFERENCES

1. NestJS Documentation. *"NestJS - A progressive Node.js framework for building efficient, reliable and scalable server-side applications."* URL: https://docs.nestjs.com/.
2. Sequelize ORM Reference. *"Sequelize is a promise-based Node.js ORM for Postgres, MySQL, MariaDB, SQLite and Microsoft SQL Server."* URL: https://sequelize.org/.
3. AWS Bedrock API Guide. *"Amazon Bedrock - Build and scale generative AI applications with foundation models."* URL: https://docs.aws.amazon.com/bedrock/.
4. Gemma 3 Open Model Family. *"Google Gemma 3 Models - Open models built from the same research and technology used to create the Gemini models."* URL: https://ai.google.dev/gemma.
5. Nodemailer Documentation. *"Nodemailer - Send emails from Node.js – easy as cake!"* URL: https://nodemailer.com/.
6. Chart.js Documentation. *"Simple yet flexible JavaScript charting for designers & developers."* URL: https://www.chartjs.org/.
7. Winston Logging Library. *"A logger for just about everything."* URL: https://github.com/winstonjs/winston/.
8. Quizminia Live Deployment. *"AI-Adaptive Learning Platform with Real-Time Difficulty Adjustment."* URL: https://quizminia.examprap.click/.